System for Artificial Intelligence

Automatic Differentiation

Siyuan Feng
Shanghai Innovation Institute

OUTLINE

01 Differentiation Methods

02 Reverse Mode Auto Diff

03 PyTorch Dynamic Autograd

01

Differentiation Methods

Numerical Differentiation

上海创智学院
Shanghai Innovation Institute

The partial derivative is defined by a limit

$$\frac{\partial f(\theta)}{\partial \theta_i} = \lim_{\epsilon \to 0} \frac{f(\theta + \epsilon e_i) - f(\theta)}{\epsilon}$$

Central difference for a sufficiently smooth function

$$\frac{\partial f(\theta)}{\partial \theta_i} = \frac{f(\theta + \epsilon e_i) - f(\theta - \epsilon e_i)}{2\epsilon} + O(\epsilon^2)$$

Finite differences have **numerical error** and high full-gradient cost

Numerical Gradient Checking

上海创智学院
Shanghai Innovation Institute

Numerical differentiation can check an automatic differentiation implementation.

$$\delta^T \nabla_\theta f(\theta) = \frac{f(\theta + \epsilon\delta) - f(\theta - \epsilon\delta)}{2\epsilon} + O(\epsilon^2)$$

Choose a unit direction δ and compare both sides within a tolerance.

Symbolic Differentiation

上海创智学院
Shanghai Innovation Institute

Use the model formula to derive gradients by sum, product and chain rules

- $\frac{\partial\left(f(\theta)+g(\theta)\right)}{\partial\theta} =$
- $\frac{\partial\left(f(\theta)\cdot g(\theta)\right)}{\partial\theta} =$
- $\frac{\partial\left(f\left(g(\theta)\right)\right)}{\partial\theta} =$

Symbolic Differentiation

上海创智学院
Shanghai Innovation Institute

Use the model formula to derive gradients by sum, product and chain rules

- $\frac{\partial\left(f(\theta)+g(\theta)\right)}{\partial\theta} = \frac{\partial\left(f(\theta)\right)}{\partial\theta} + \frac{\partial\left(g(\theta)\right)}{\partial\theta}$
- $\frac{\partial\left(f(\theta)\cdot g(\theta)\right)}{\partial\theta} = g(\theta)\frac{\partial\left(f(\theta)\right)}{\partial\theta} + f(\theta)\frac{\partial\left(g(\theta)\right)}{\partial\theta}$
- $\frac{\partial\left(f\left(g(\theta)\right)\right)}{\partial\theta} = \frac{\partial\left(f\left(g(\theta)\right)\right)}{\partial g(\theta)}\frac{\partial\left(g(\theta)\right)}{\partial\theta}$

Symbolic Differentiation

上海创智学院
Shanghai Innovation Institute

A naive method can result in repeated computation

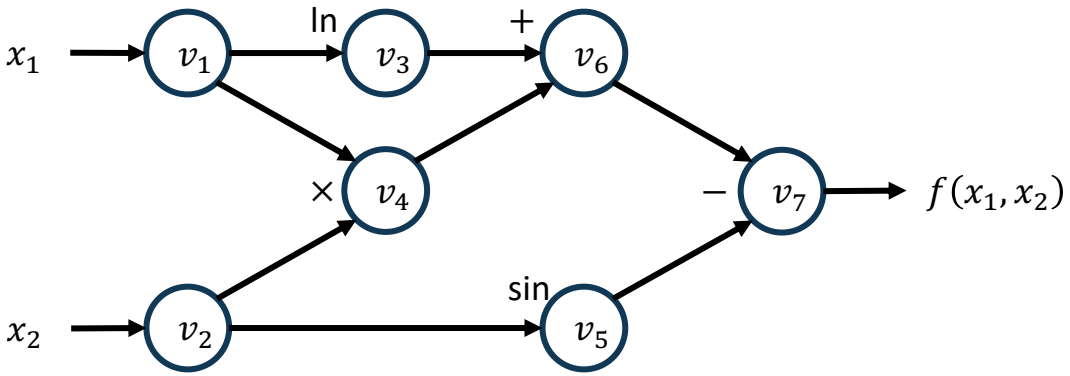
Example:

$$f(\theta) = \prod_{i=1}^{n} \theta_i \,, \qquad \frac{\partial f(\theta)}{\partial \theta_k} = \prod_{i \neq k}^{n} \theta_i$$

For independent products: $n(n-2)$ multiplications for all partial derivatives

Recap: Computational Graph

$$f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$$

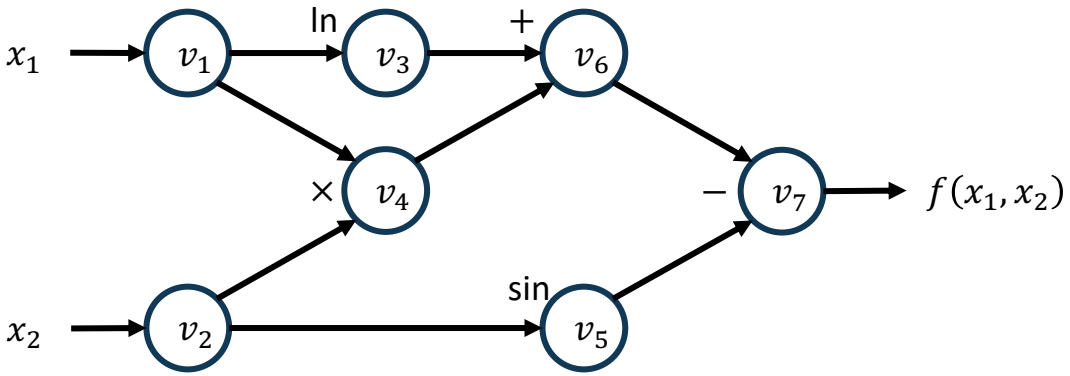


Each node represents an input or intermediate value. Edges represent data dependencies.

Example based on A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind. Automatic differentiation in machine learning: a survey. JMLR 2018

Forward Mode Automatic Differentiation

$$f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$$



$$v_1 = x_1 = 2$$
$$v_2 = x_2 = 5$$
$$v_3 = \ln(v_1) = 0.693$$
$$v_4 = v_1 \times v_2 = 10$$
$$v_5 = \sin(v_2) = -0.959$$
$$v_6 = v_3 + v_4 = 10.693$$
$$v_7 = v_6 - v_5 = 11.652$$

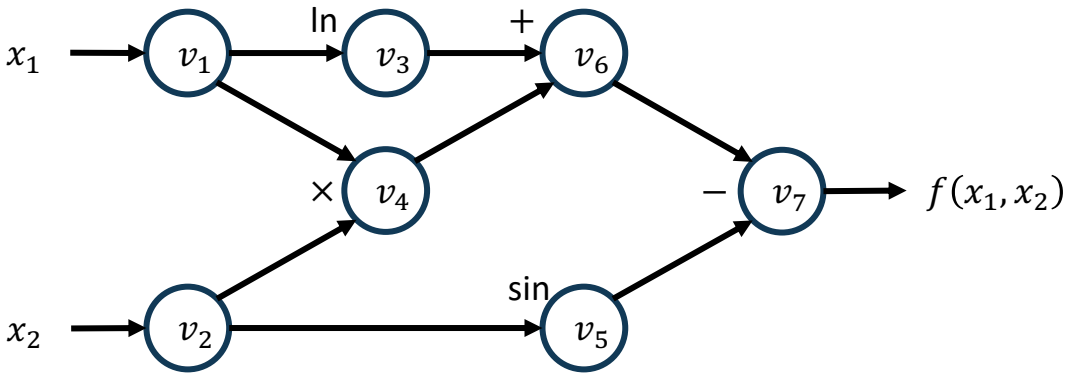
Define $\dot{v}_i = \frac{\partial v_i}{\partial x_1}$

We can then compute the $\dot{v}_i$ iteratively in the forward topological order of the computational graph

$$\dot{v}_1 = 1$$
$$\dot{v}_2 = 0$$
$$\dot{v}_3 = \frac{\dot{v}_1}{v_1} = 0.5$$
$$\dot{v}_4 = \dot{v}_1 v_2 + \dot{v}_2 v_1 = 5$$
$$\dot{v}_5 = \dot{v}_2 \cos(v_2) = 0$$
$$\dot{v}_6 = \dot{v}_3 + \dot{v}_4 = 5.5$$
$$\dot{v}_7 = \dot{v}_6 - \dot{v}_5 = 5.5$$
$$\frac{\partial f(x_1, x_2)}{\partial x_1} = \dot{v}_7 = 5.5$$

Forward Mode Automatic Differentiation

$$f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$$



$$v_1 = x_1 = 2$$
$$v_2 = x_2 = 5$$
$$v_3 = \ln(v_1) = 0.693$$
$$v_4 = v_1 \times v_2 = 10$$
$$v_5 = \sin(v_2) = -0.959$$
$$v_6 = v_3 + v_4 = 10.693$$
$$v_7 = v_6 - v_5 = 11.652$$

Define $\dot{v}_i = \frac{\partial v_i}{\partial x_2}$

$$\dot{v}_1 = 0$$
$$\dot{v}_2 = 1$$
$$\dot{v}_3 = \frac{\dot{v}_1}{v_1} = 0$$
$$\dot{v}_4 = \dot{v}_1 v_2 + \dot{v}_2 v_1 = 2$$
$$\dot{v}_5 = \dot{v}_2 \cos(v_2) = 0.284$$
$$\dot{v}_6 = \dot{v}_3 + \dot{v}_4 = 2$$
$$\dot{v}_7 = \dot{v}_6 - \dot{v}_5 = 1.716$$

$$\frac{\partial f(x_1, x_2)}{\partial x_2} = \dot{v}_7 = 1.716$$

Limitations of Forward Mode

上海创智学院
Shanghai Innovation Institute

- For $f: \mathbb{R}^n \to \mathbb{R}^k$, n forward passes compute the full Jacobian.

- For a scalar loss, $k = 1$. The input dimension n may be large.

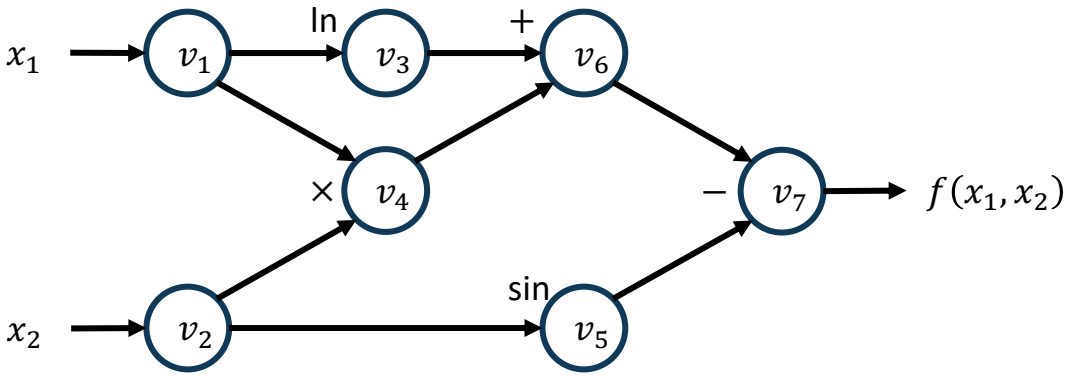
- Full Jacobian: forward $O(nC_f)$; reverse $O(kC_f)$.
- Here C_f is the cost of one function evaluation.

02

Reverse Mode Auto Diff

Reverse Mode Automatic Differentiation

$$f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$$



$$v_1 = x_1 = 2$$
$$v_2 = x_2 = 5$$
$$v_3 = \ln(v_1) = 0.693$$
$$v_4 = v_1 \times v_2 = 10$$
$$v_5 = \sin(v_2) = -0.959$$
$$v_6 = v_3 + v_4 = 10.693$$
$$v_7 = v_6 - v_5 = 11.652$$

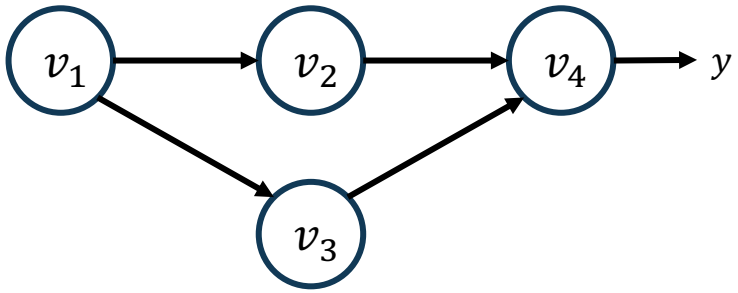
Scalar output: one backward pass after forward

Define $\bar{v}_i = \frac{\partial f(x_1, x_2)}{\partial v_i}$, while forward mode is $\dot{v}_i = \frac{\partial v_i}{\partial x_1}$

We can then compute the $\bar{v}_i$ iteratively in the **reverse** topological order of the computational graph

$$\bar{v}_7 = \frac{\partial f(x_1, x_2)}{\partial v_7} = 1$$
$$\bar{v}_6 = \bar{v}_7 \cdot \frac{\partial v_7}{\partial v_6} = \bar{v}_7 \cdot 1 = 1$$
$$\bar{v}_5 = \bar{v}_7 \cdot \frac{\partial v_7}{\partial v_5} = \bar{v}_7 \cdot -1 = -1$$
$$\bar{v}_4 = \bar{v}_6 \cdot \frac{\partial v_6}{\partial v_4} = \bar{v}_6 \cdot 1 = 1$$
$$\bar{v}_3 = \bar{v}_6 \cdot \frac{\partial v_6}{\partial v_3} = \bar{v}_6 \cdot 1 = 1$$
$$\bar{v}_2 = \bar{v}_5 \cdot \frac{\partial v_5}{\partial v_2} + \bar{v}_4 \cdot \frac{\partial v_4}{\partial v_2} = \bar{v}_5 \cdot \cos(v_2) + \bar{v}_4 \cdot v_1 = 1.716$$
$$\bar{v}_1 = \bar{v}_4 \cdot \frac{\partial v_4}{\partial v_1} + \bar{v}_3 \cdot \frac{\partial v_3}{\partial v_1} = \bar{v}_4 \cdot v_2 + \bar{v}_3 \cdot \frac{1}{v_1} = 5.5$$

Derivation for Branches



In this case, v_1 is being used in multiple pathways (i.e., v_2 and v_3)

y can be written in form of $y = f(v_2, v_3)$, hence

$$\bar{v}_1 = \frac{\partial y}{\partial v_1} = \frac{\partial f(v_2, v_3)}{\partial v_1} = \frac{\partial f(v_2, v_3)}{\partial v_2} \cdot \frac{\partial v_2}{\partial v_1} + \frac{\partial f(v_2, v_3)}{\partial v_3} \cdot \frac{\partial v_3}{\partial v_1} = \bar{v}_2 \cdot \frac{\partial v_2}{\partial v_1} + \bar{v}_3 \cdot \frac{\partial v_3}{\partial v_1}$$

Define partial adjoint $\bar{v}_{i \to j} = \bar{v}_j \cdot \frac{\partial v_j}{\partial v_i}$ for each input output node pair i and j, we have

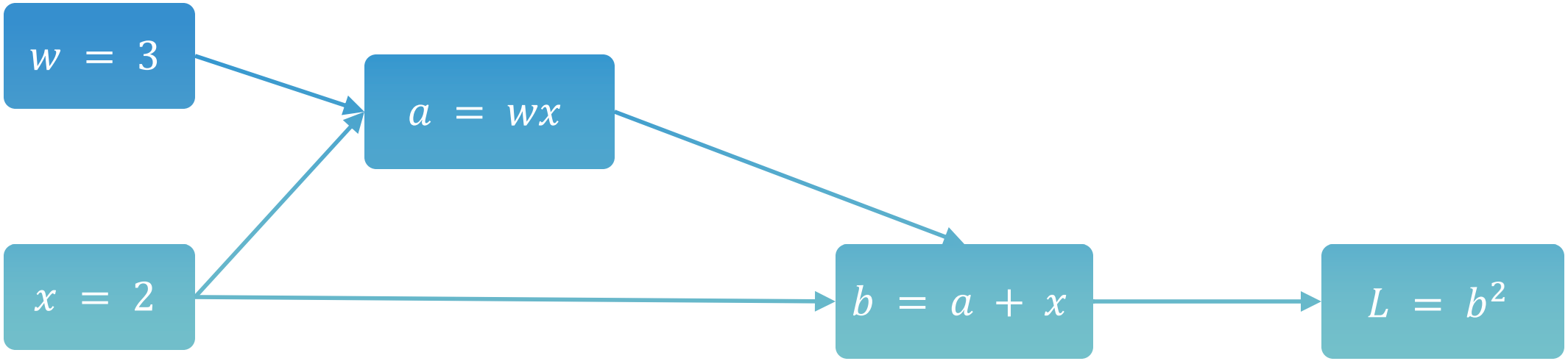
$$\bar{v}_i = \sum_{j \in adjoints(i)} \bar{v}_{i \to j}$$

We can compute partial adjoints separately then sum them together

Quick Check: A Residual Path

上海创智学院
Shanghai Innovation Institute

3 minutes: compute the loss and both gradients.

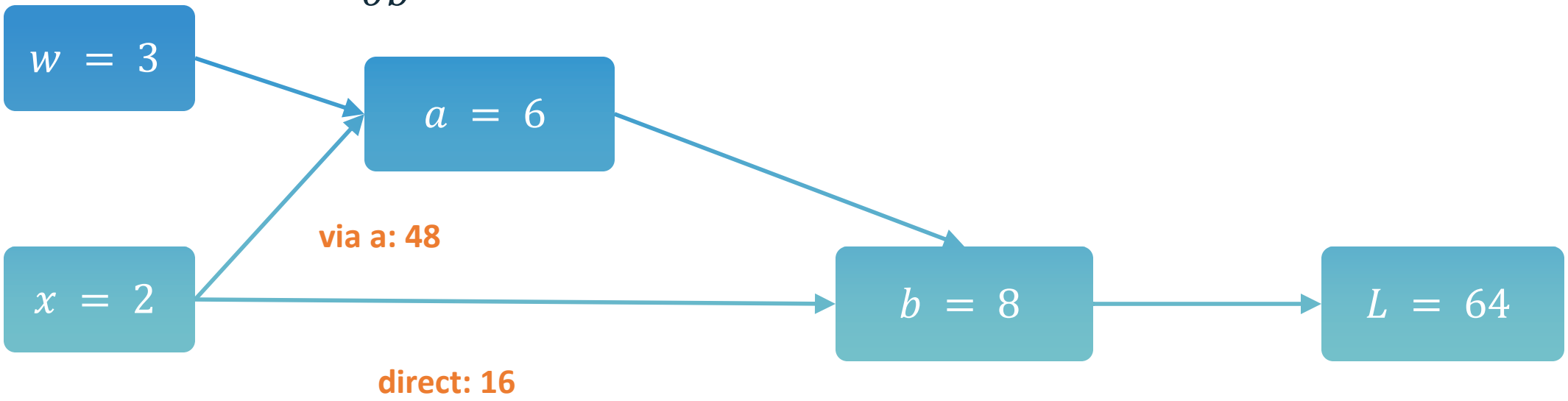


$$L = ? \quad \frac{\partial L}{\partial x} = ? \quad \frac{\partial L}{\partial w} = ?$$

Why does x receive two gradient contributions?

Residual Path: Gradient Check

$$\frac{\partial L}{\partial b} = 2b = 16$$



$$\frac{\partial L}{\partial x} = 16w + 16 = 64$$

$$\frac{\partial L}{\partial w} = 16x = 32$$

Multiply along each path. Add contributions at a shared input.

Reverse AD Algorithm (Pseudocode)

上海创智学院
Shanghai Innovation Institute

```
def gradient(out, wrt):
    node_to_grad = {out: [1]}

    for i in reverse_topo_order(out):
        v̄_i = Σ_j v̄_{i→j} =sum(node_to_grad[i])

        for k in inputs[i]:
            compute v̄_{k→i} = v̄_i ∂v_i/∂v_k
            append v̄_{k→i} to node_to_grad[k]
    return adjoints of wrt
```

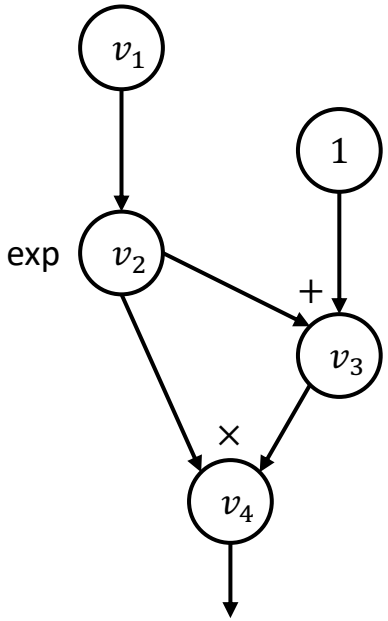
node_to_grad: partial-adjoint lists; missing keys start with []

Sum up partial adjoints

Propagates partial adjoint to its input node

Reverse Mode AD by Extending Computational Graph


```
def gradient(out, wrt):
    node_to_grad = {out: [1]}  # default []
    for i in reverse_topo_order(out):
        v̄_i = Σ_j v̄_{i→j} =sum(node_to_grad[i])
        for k in inputs[i]:
            compute v̄_{k→i} = v̄_i ∂v_i/∂v_k
            append v̄_{k→i} to node_to_grad[k]
    return adjoints of wrt
```





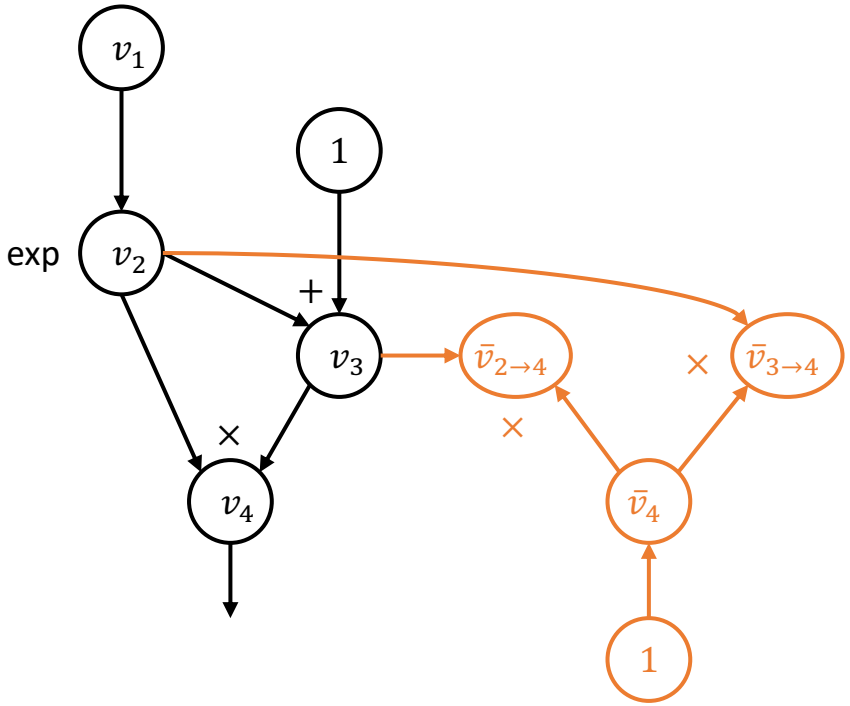
$i = 4$

node_to_grad:{
    4: [1]
}

computed $\bar{v}_i$:{
    $\bar{v}_4$
}

Reverse Mode AD by Extending Computational Graph


```
def gradient(out, wrt):
    node_to_grad = {out: [1]}  # default []
    for i in reverse_topo_order(out):
        v̄_i = Σ_j v̄_{i→j} =sum(node_to_grad[i])
        for k in inputs[i]:
            compute v̄_{k→i} = v̄_i ∂v_i/∂v_k
            append v̄_{k→i} to node_to_grad[k]
    return adjoints of wrt
```



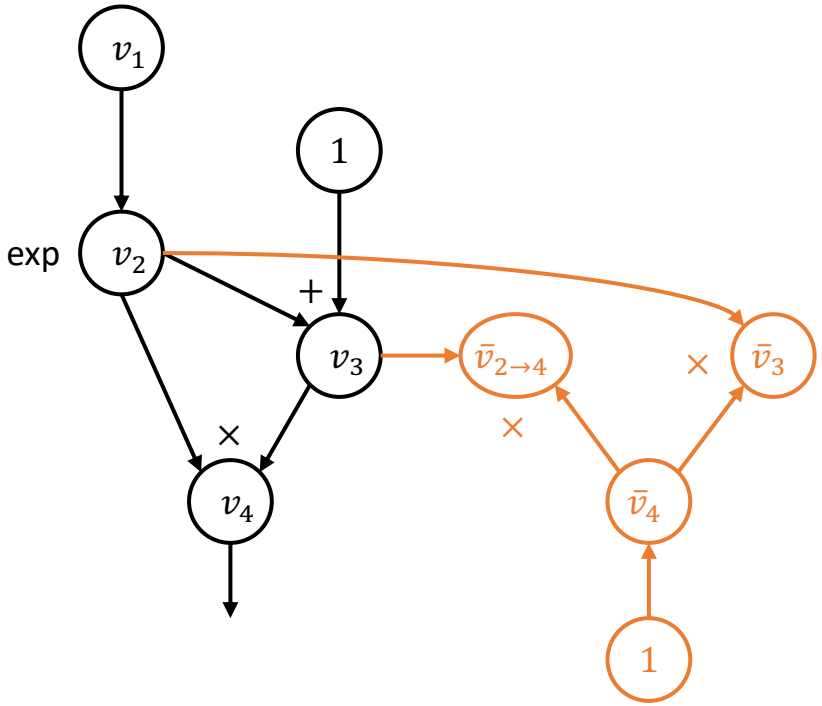
$i = 4$

node_to_grad:{
    2: $[\bar{v}_{2\to4}]$
    3: $[\bar{v}_{3\to4}]$
    4: [1]
}

computed $\bar{v}_i$:{
    $\bar{v}_4$
}

Reverse Mode AD by Extending Computational Graph

```
def gradient(out, wrt):
    node_to_grad = {out: [1]}  # default []
    for i in reverse_topo_order(out):
        v̄_i = Σ_j v̄_{i→j} =sum(node_to_grad[i])
        for k in inputs[i]:
            compute v̄_{k→i} = v̄_i ∂v_i/∂v_k
            append v̄_{k→i} to node_to_grad[k]
    return adjoints of wrt
```



$i = 3$

node_to_grad:{
    2: $[\bar{v}_{2\to4}]$
    3: $[\bar{v}_{3\to4}]$
    4: [1]
}

computed $\bar{v}_i$:{
    $\bar{v}_3$
    $\bar{v}_4$
}

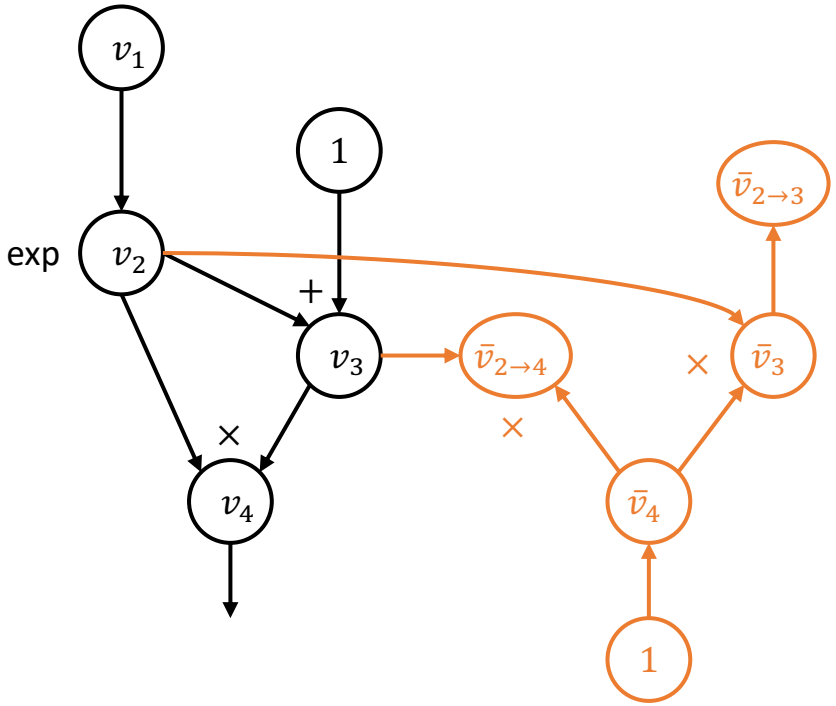
Reverse Mode AD by Extending Computational Graph


```
def gradient(out, wrt):
    node_to_grad = {out: [1]}  # default []
    for i in reverse_topo_order(out):
        v̄_i = Σ_j v̄_{i→j} =sum(node_to_grad[i])
        for k in inputs[i]:
            compute v̄_{k→i} = v̄_i ∂v_i/∂v_k
            append v̄_{k→i} to node_to_grad[k]
    return adjoints of wrt
```

$i = 3$

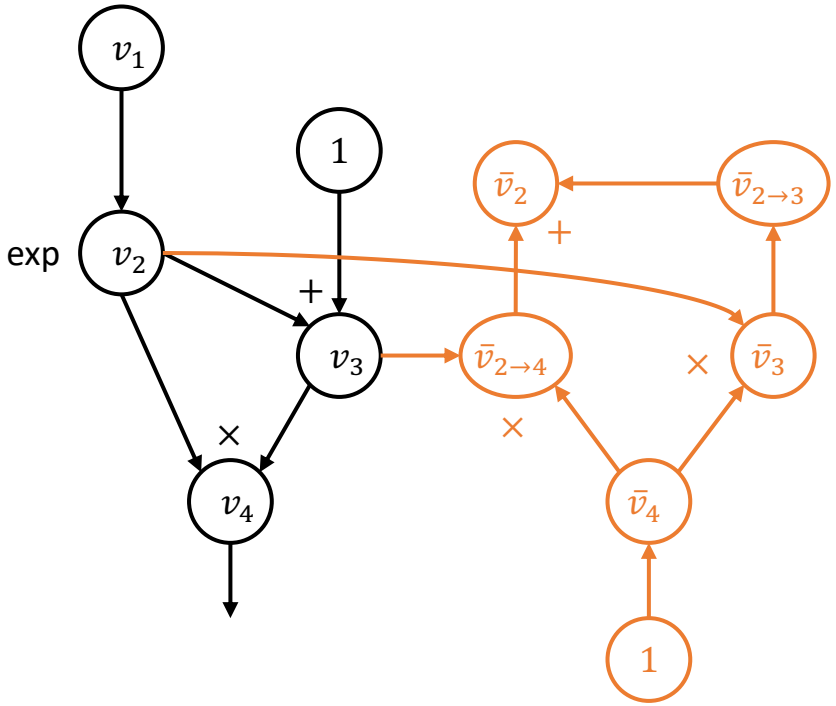
node_to_grad:{
  2: $[\bar{v}_{2\to4}, \bar{v}_{2\to3}]$
  3: $[\bar{v}_{3\to4}]$
  4: [1]
}

computed $\bar{v}_i$:{
  $\bar{v}_3$
  $\bar{v}_4$
}



Reverse Mode AD by Extending Computational Graph

```
def gradient(out, wrt):
    node_to_grad = {out: [1]}  # default []
    for i in reverse_topo_order(out):
        v̄_i = Σ_j v̄_{i→j} =sum(node_to_grad[i])
        for k in inputs[i]:
            compute v̄_{k→i} = v̄_i ∂v_i/∂v_k
            append v̄_{k→i} to node_to_grad[k]
    return adjoints of wrt
```



$i = 2$

node_to_grad:{
  2: $[\bar{v}_{2\to4}, \bar{v}_{2\to3}]$
  3: $[\bar{v}_{3\to4}]$
  4: [1]
}

computed $\bar{v}_i$:{
  $\bar{v}_2$
  $\bar{v}_3$
  $\bar{v}_4$
}

Reverse Mode AD by Extending Computational Graph


```
def gradient(out, wrt):
    node_to_grad = {out: [1]}  # default []
    for i in reverse_topo_order(out):
        v̄_i = Σ_j v̄_{i→j} =sum(node_to_grad[i])
        for k in inputs[i]:
            compute v̄_{k→i} = v̄_i ∂v_i/∂v_k
            append v̄_{k→i} to node_to_grad[k]
    return adjoints of wrt
```

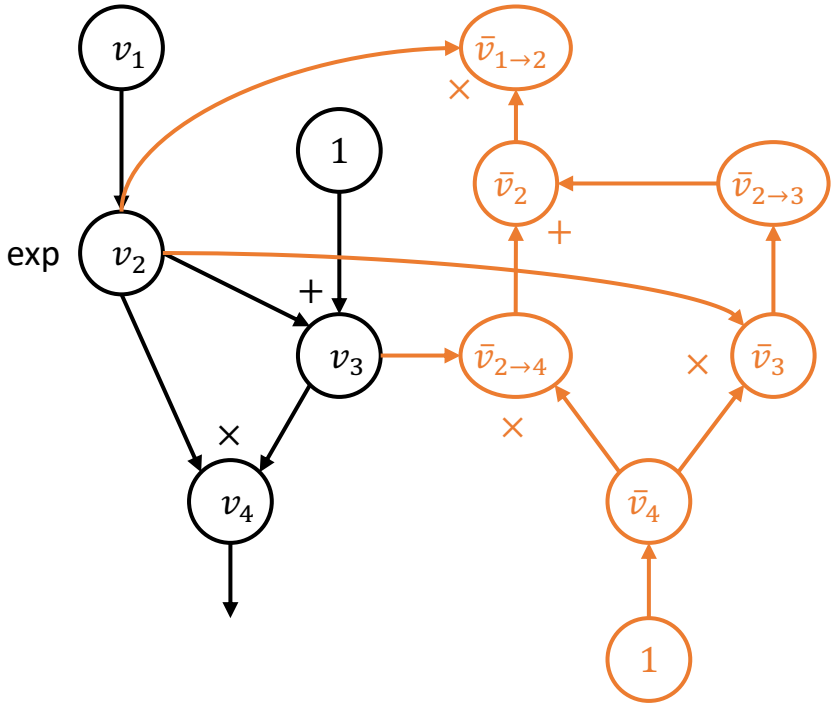
$i = 2$

node_to_grad:{
- 1: $[\bar{v}_{1\to 2}]$
- 2: $[\bar{v}_{2\to 4}, \bar{v}_{2\to 3}]$
- 3: $[\bar{v}_{3\to 4}]$
- 4: $[1]$

}

computed $\bar{v}_i$:{
- $\bar{v}_2$
- $\bar{v}_3$
- $\bar{v}_4$

}



Reverse Mode AD by Extending Computational Graph


```
def gradient(out, wrt):
    node_to_grad = {out: [1]}  # default []
    for i in reverse_topo_order(out):
        v̄_i = Σ_j v̄_{i→j} =sum(node_to_grad[i])
        for k in inputs[i]:
            compute v̄_{k→i} = v̄_i ∂v_i/∂v_k
            append v̄_{k→i} to node_to_grad[k]
    return adjoints of wrt
```

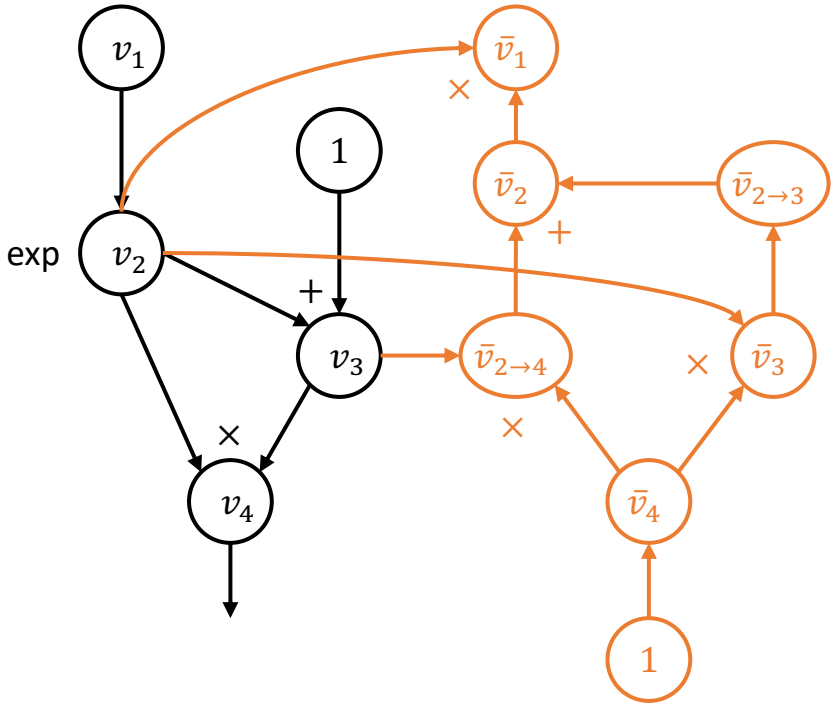
$i = 1$

node_to_grad:{
- 1: $[\bar{v}_{1\to 2}]$
- 2: $[\bar{v}_{2\to 4}, \bar{v}_{2\to 3}]$
- 3: $[\bar{v}_{3\to 4}]$
- 4: $[1]$

}

computed $\bar{v}_i$:{
- $\bar{v}_1$
- $\bar{v}_2$
- $\bar{v}_3$
- $\bar{v}_4$

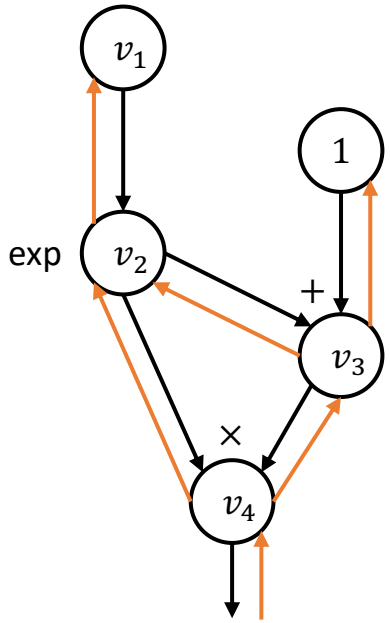
}



Compute in-place vs. Extend Computational Graph

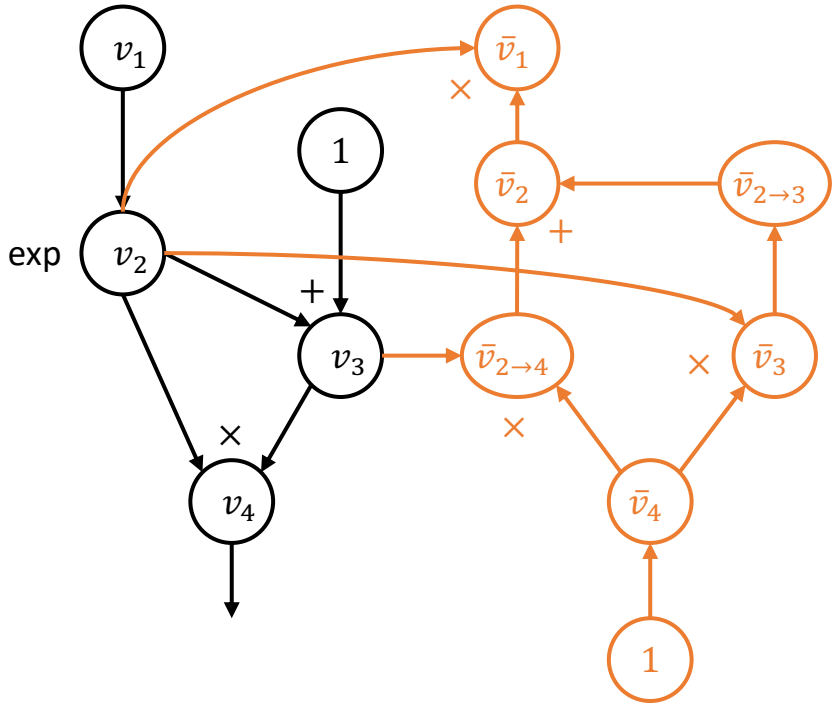
上海创智学院
Shanghai Innovation Institute

Reverse mode AutoDiff by computing in-place



- Run backward rules on the forward graph.
- Accumulate numerical adjoints.

Reverse mode AutoDiff by extending computational graph



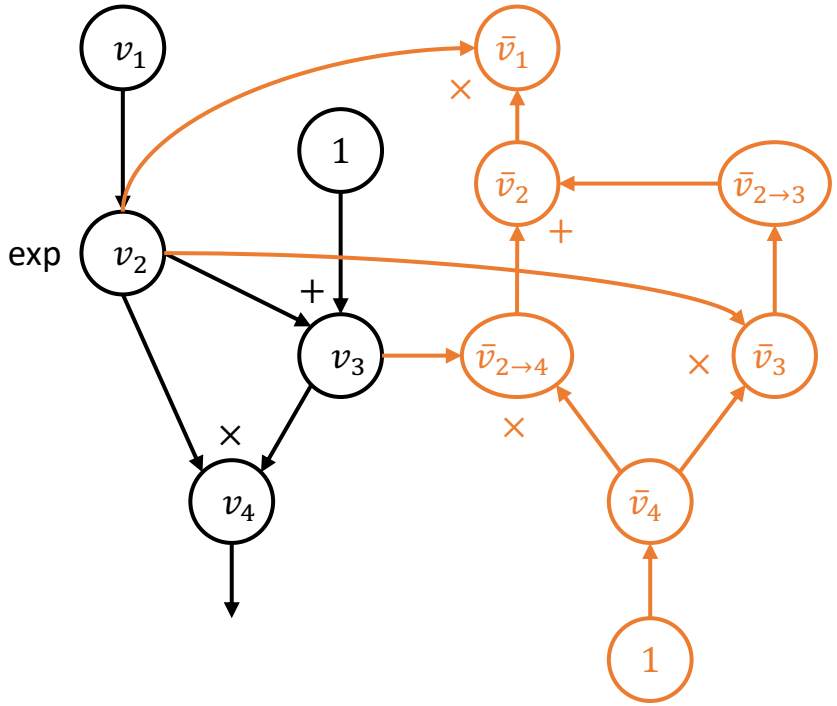
- Represent gradients as graph operations.
- Enables transforms and higher derivatives.

Why Extend Computational Graph

上海创智学院
Shanghai Innovation Institute

- Unified execution runtime
- Higher-order differentiation
  - Record gradient operations as a differentiable graph
  - In PyTorch, use create_graph=True to differentiate through gradient computations

Reverse mode AutoDiff by extending computational graph



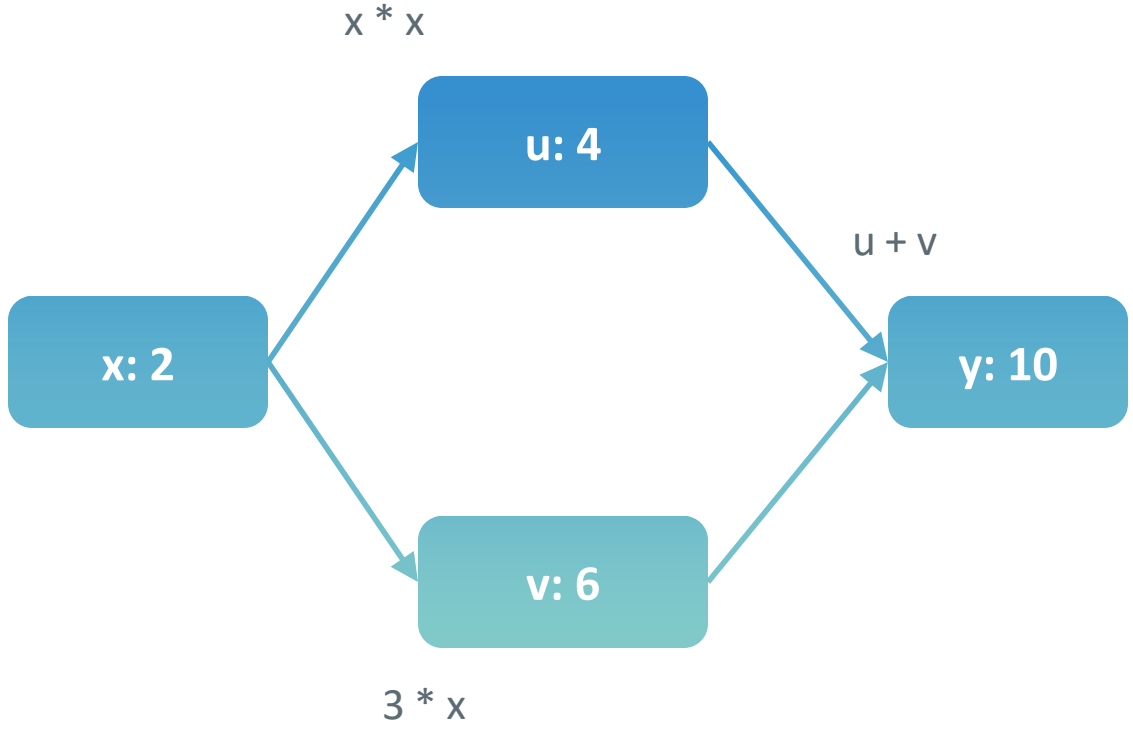
- Represent gradients as graph operations.
- Enables transforms and higher derivatives.

03

PyTorch Dynamic Autograd

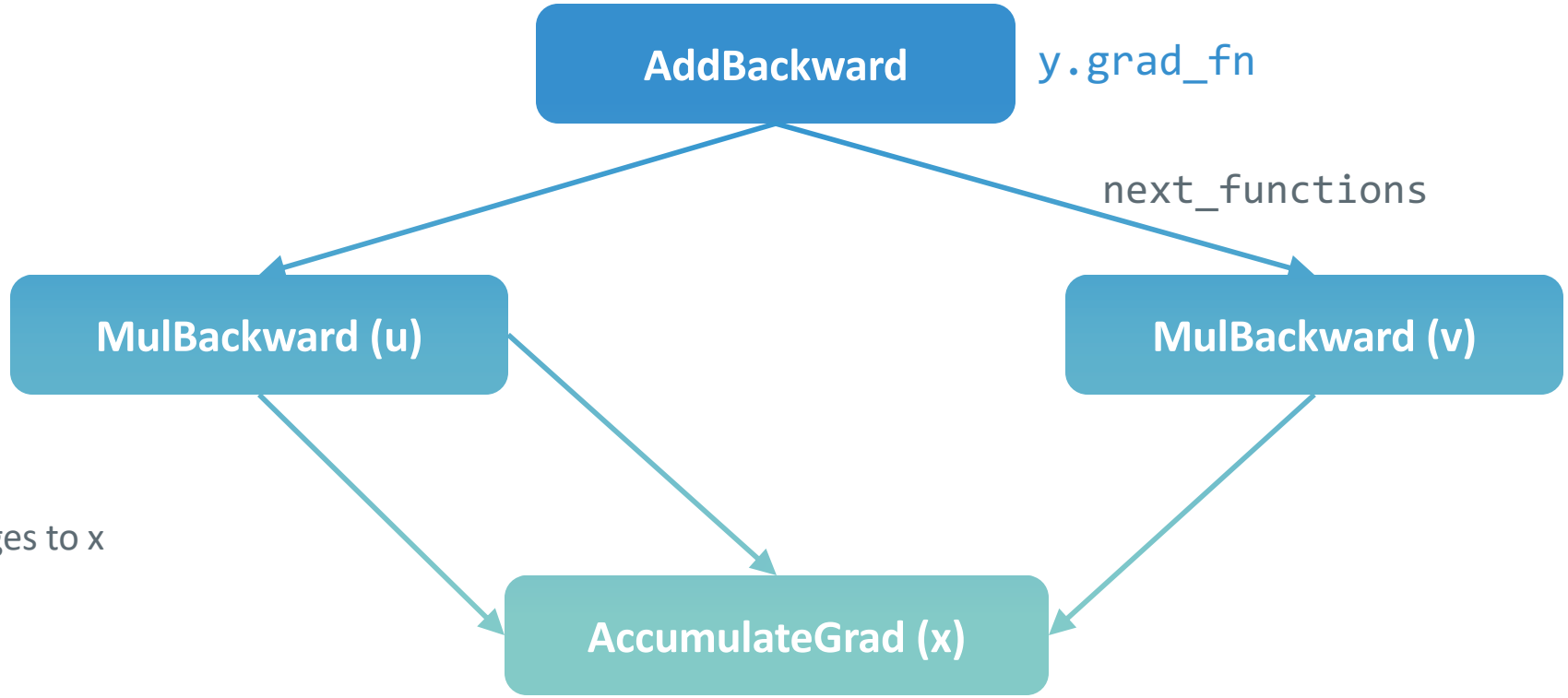
Forward Execution Builds the Graph

```
import torch
x = torch.tensor(2.,
    requires_grad=True)
u = x * x
v = 3 * x
y = u + v
```



Forward: tensor values and backward nodes. x.grad is still None.

Tensors and Backward Nodes



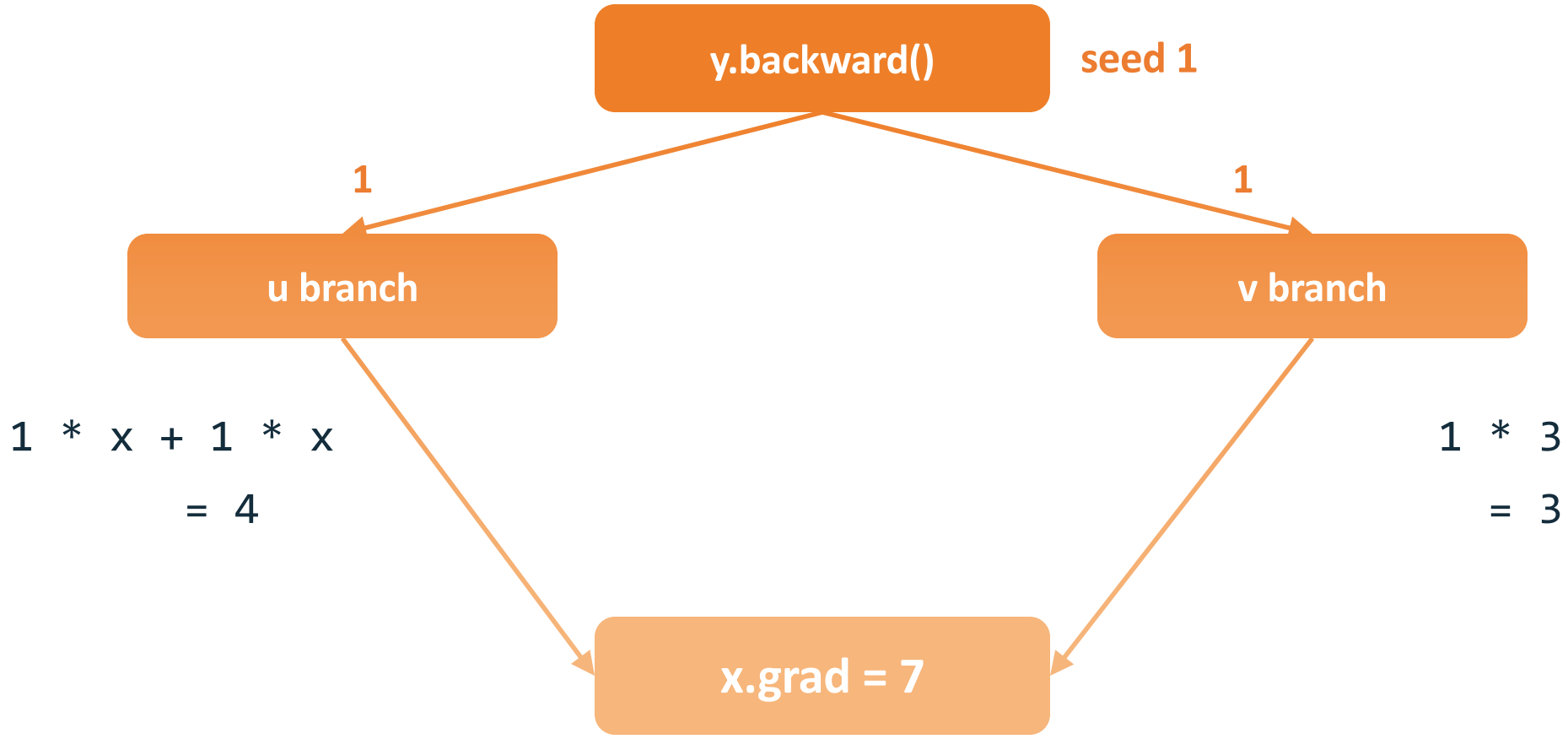
two input edges to x

`x: leaf, grad_fn=None`

`u: non-leaf, grad_fn=MulBackward`

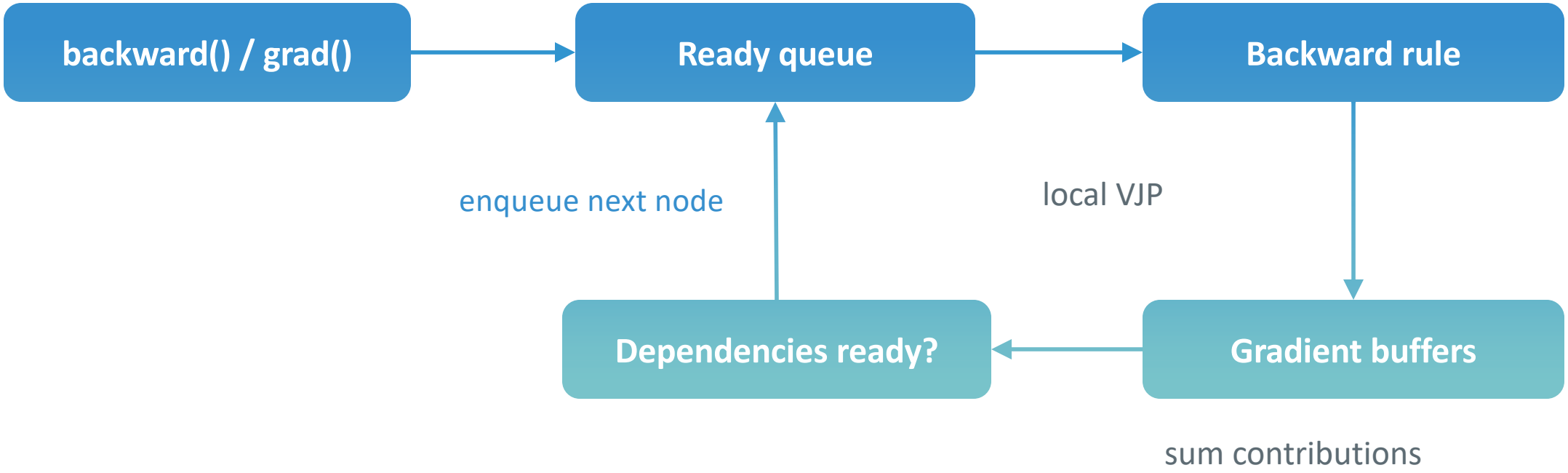
After backward: x.grad stores the result. Use u.retain_grad() to retain u.grad.

Backward: Local Products and Sums



Each input position contributes. Shared inputs receive the sum.

Autograd Engine Execution



`backward(): accumulate into .grad`

`grad(): return requested gradients`

Dynamic Control Flow

上海创智学院
Shanghai Innovation Institute

```
def f(x):
    if x.item() > 0:
        return x * x
    return 3 * x
```

Only the executed branch is recorded.

The Python condition itself is not differentiated.

Run A: x = 2



output: 4 gradient: 4

Run B: x = -2



output: -6 gradient: 3

Vector Outputs and VJPs

上海创智学院
Shanghai Innovation Institute

```
x = torch.tensor([2., 3.],
    requires_grad=True)
y = x * x
v = torch.tensor([1., 2.])
g, = torch.autograd.grad(
    y, x, grad_outputs=v)
```

v[0] = 1

weight of y[0]

v[1] = 2

weight of y[1]

$$J^T v = [4, 12]^T$$

grad_outputs supplies the upstream gradient. A scalar loss uses seed 1.

Gradient Accumulation in Training

上海创智学院
Shanghai Innovation Institute

Fresh graph, persistent .grad

```
x = torch.tensor(2.,
    requires_grad=True)
for _ in range(2):
    y = x*x + 3*x
    y.backward()
    print(x.grad.item())
    # 7.0, then 14.0
```

A usual training iteration

```
optimizer.zero_grad(
    set_to_none=True)
loss = criterion(model(x), target)
loss.backward()
optimizer.step()
```

backward computes gradients. step updates parameters. zero_grad resets buffers.

Saved Tensors and Graph Lifetime

上海创智学院
Shanghai Innovation Institute




```
y = x * x
y.backward()
y.backward()  # saved state is gone
```

retain_graph=True

Keep state for another traversal

In-place edits to saved values can invalidate backward.

A visible grad_fn does not guarantee that its saved tensors still exist.

Controlling Gradient Tracking

上海创智学院
Shanghai Innovation Institute

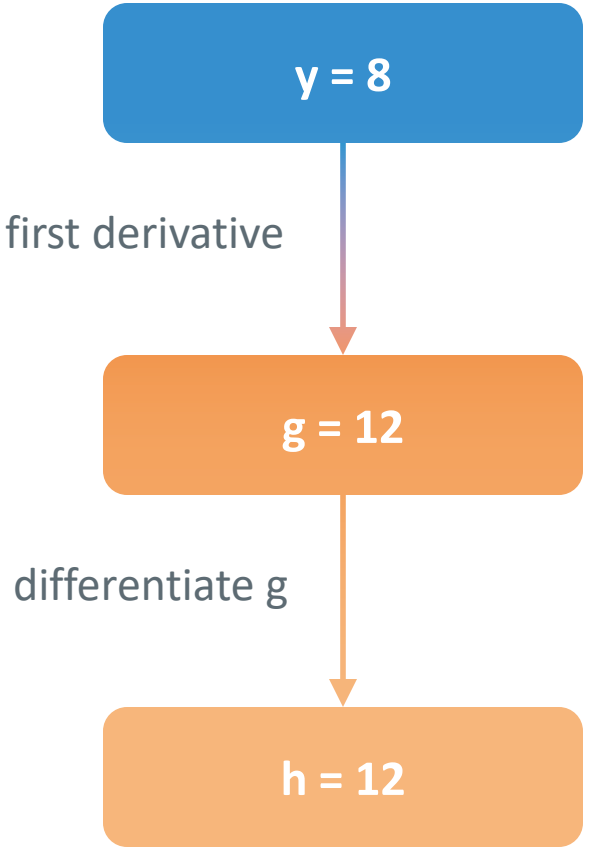
`p.requires_grad_(False)`	**Freeze a leaf parameter** Other inputs may still need gradients.
`with torch.no_grad():`	**Disable recording in a region** Useful for inference or parameter updates.
`z = y.detach()`	**Cut the connection to prior history** The new tensor shares storage with y.
`model.eval()`	**Change module behavior** Gradient tracking remains independent.

Higher-Order Derivatives

上海创智学院
Shanghai Innovation Institute

```
x = torch.tensor(2.,
    requires_grad=True)
y = x ** 3
g, = torch.autograd.grad(
    y, x, create_graph=True)
h, = torch.autograd.grad(g, x)
```

$$g = 3x^2 = 12, \qquad h = 6x = 12$$



create_graph records derivative operations. retain_graph keeps existing state.

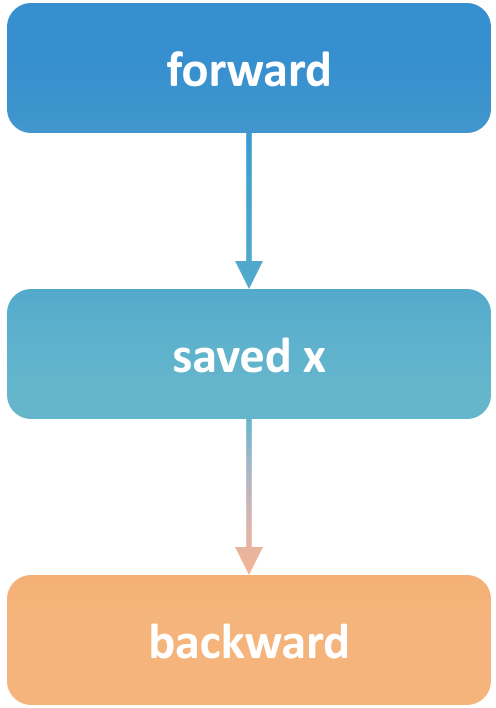
Custom Autograd Functions

上海创智学院
Shanghai Innovation Institute

```
class Square(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x):
        ctx.save_for_backward(x)
        return x * x

    @staticmethod
    def backward(ctx, grad_out):
        x, = ctx.saved_tensors
        return grad_out * 2 * x

y = Square.apply(x)
```



upstream gradient

The backward rule returns a local VJP. Check it with torch.autograd.gradcheck.

Exercise: Predict the Gradient

上海创智学院
Shanghai Innovation Institute

```
x = torch.tensor(2.,
    requires_grad=True)
a = x * x
y = a + 3 * x
y.backward()
```

A **Insert a = a.detach()**

What is x.grad?

B **Call a.retain_grad()**

What are a.grad and x.grad?

C **Recompute and backward twice**

What if .grad is never cleared?

Sketch the active paths first. Then add their gradient contributions.

Quick Check: Attention Forward

One query, two key/value tokens. No mask or dropout.

$$Q = \begin{bmatrix} 1 & 1 \end{bmatrix} \qquad K = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \qquad V = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

Q: 1 × 2 **K: 2 × 2** **V: 2 × 2**

$$S = \frac{QK^T}{\sqrt{2}} \rightarrow P = softmax(S) \rightarrow O = PV$$

$$T = \begin{bmatrix} 0 & 1 \end{bmatrix} \quad L = \frac{1}{2}\|O - T\|^2$$

Find P, O, L, and the gradient of V.

Attention: Output and Value Gradients

上海创智学院
Shanghai Innovation Institute

$$S = [0 \quad 0]$$

$$P = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \end{bmatrix}$$

$$O = [1 \quad 1]$$

$$L = \frac{1}{2}$$

$$G = \frac{\partial L}{\partial O} = [1 \quad 0]$$

$$dV = P^T G = \begin{bmatrix} \frac{1}{2} & 0 \\ \frac{1}{2} & 0 \end{bmatrix}$$

$$Notation: \; dX = \frac{\partial L}{\partial X}$$

Each value row receives its attention weight multiplied by G.

Quick Check: Attention Backward

上海创智学院
Shanghai Innovation Institute

$$Q = [1 \quad 1]$$

$$K = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

Given

$$P = \left[\frac{1}{2} \quad \frac{1}{2}\right] \quad dP = GV^T = [2 \quad 0]$$

Softmax backward rule

$$dS_i = P_i(dP_i - \sum_j P_j dP_j)$$

$$dQ = \frac{dS\,K}{\sqrt{2}}$$

$$dK = \frac{dS^T Q}{\sqrt{2}}$$

Find dS (1 × 2), dQ (1 × 2), and dK (2 × 2).

Attention: Query and Key Gradients

上海创智学院
Shanghai Innovation Institute

$$dS = \begin{bmatrix} \frac{1}{2} & -\frac{1}{2} \end{bmatrix}$$

Query gradient: 1 × 2

$$dQ = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \end{bmatrix}$$

Key gradient: 2 × 2

$$dK = \frac{1}{2\sqrt{2}} \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}$$

$$Check: \; dS_1 + dS_2 = 0$$

Equal attention weights can still have nonzero gradients.

Discussion: Save or Recompute?

上海创智学院
Shanghai Innovation Institute

Self-attention: N queries and N keys, one batch, one head.

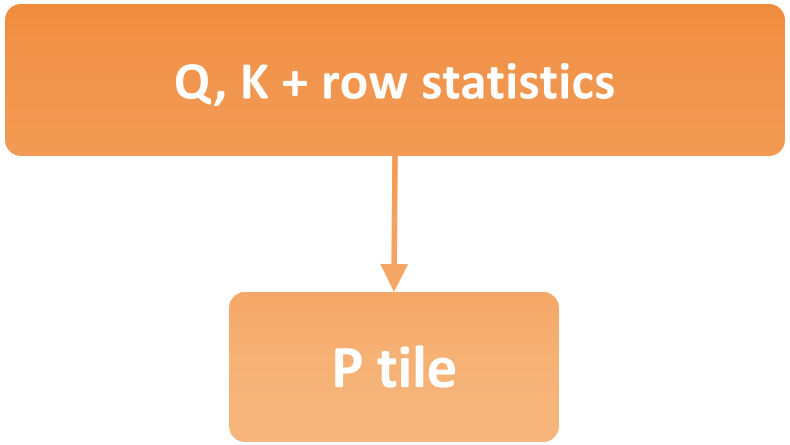
Keep the full probability matrix

$$P: \ N \times N$$

$$4096^2 \times 2 \ bytes \ = \ 32 \ MiB$$

P only, assuming FP16 storage. Other tensors excluded.

Recompute probabilities in tiles



Extra computation
Less stored state

If N doubles, how does storage grow? Must backward keep the full P?

Acknowledgement

The development of this course, including its structure, content, and accompanying presentation slides, has been significantly influenced and inspired by the excellent work of instructors and institutions who have shared their materials openly. We wish to extend our sincere acknowledgement and gratitude to the following courses, which served as invaluable references and a source of pedagogical inspiration:

- Machine Learning Systems[15-442/15-642], by **Tianqi Chen** and **Zhihao Jia** at **CMU**.
- Advanced Topics in Machine Learning (Systems)[CS6216], by **Yao Lu** at **NUS**

While these materials provided a foundational blueprint and a wealth of insightful examples, all content herein has been adapted, modified, and curated to meet the specific learning objectives of our curriculum. Any errors, omissions, or shortcomings found in these course materials are entirely our own responsibility. We are profoundly grateful for the contributions of the educators listed above, whose dedication to teaching and knowledge-sharing has made the creation of this course possible.

System for Artificial Intelligence

Thanks

Siyuan Feng
Shanghai Innovation Institute